第一章　面向上行实时业务随机到达场景的低时延高能效资源调度机制

1.1　引言

工业无线网络的上行链路承载着现场终端的状态感知、工艺参数与异常告警数据，是支撑工业闭环控制与智能决策的核心环节 [1]。实际生产中，上行实时业务流由周期采样流与事件触发流叠加而成，呈现出显著的非平稳特征：常规状态下，传感器按固定周期上报数据形成平稳的负载；而设备故障、状态突变等突发情况会引发告警信息的瞬时涌入，形成流量尖峰。以汽车焊装车间为例，单点设备故障可触发周边关联设备同步上报异常，导致上行数据包在毫秒级内激增数倍，此类流量突发事件往往成为网络运行风险的核心诱因。若调度机制缺乏极端事件的定量约束，即便系统平均负载稳定，仍可能出现任务排队时延远超阈值的情况，进而引发控制指令延迟、设备非计划停机，甚至产线中断等严重后果。

同时，工业物联网终端普遍受到能量容量与计算能力的双重约束，仅依赖本地处理难以兼顾低时延执行与长期续航需求[2]。移动边缘计算（MEC）通过任务卸载至边缘服务器，成为解决该问题的关键技术。在 MEC 使能的工业无线网络中，终端可动态调整本地 CPU 频率并按需发起卸载请求，但不同工业任务的 QoS 需求存在显著异构性：故障预警与实时控制类业务通常要求毫秒级端到端时延与超高传输可靠性，而环境监测与统计分析类业务则更关注资源利用效率与与终端能耗效率[3]。上述差异化需求叠加随机突发到达引发的资源需求波动，使系统调度面临挑战：若资源分配不足，关键业务可能发生违约；若为极端场景预留过多资源，则在常态运行下造成带宽与能量浪费[4]。因此，在随机到达与异构 QoS 需求并存的条件下，保障关键任务时延确定性的同时实现系统能效优化，成为上行资源调度的核心问题。

现有工业边缘计算与任务卸载研究多以时间平均性能为优化目标，如平均能耗、平均时延等指标[5][6]，此类方法虽能刻画系统长期运行趋势，却难以反映时延分布的尾部特性。在随机突发到达条件下，即便平均负载可控，队列仍可能在短时间内迅速积压，引发极端排队拥塞事件。对于工业任务关键型应用，核心需求是严格约束时延违约概率，而非仅保障平均性能[7]。相比之下，引入硬时延约束虽可保障最坏情况性能，却易导致资源利用率下降、队列稳定性受损，难以兼顾效率与可靠性[8]。因此，有必要构建一种介于均值优化与硬约束保障之间的统计型风险控制机制，在可接受的违约概率水平下抑制尾部时延事件。由于上行随机突发到达引发的排队时延通常呈现长尾分布特征，极值理论（Extreme Value

Theory, EVT）为尾部行为建模提供了有效工具[9]。通过峰值超阈值（Peaks-Over-Threshold, POT）方法将超阈值样本拟合为广义帕累托分布，可建立队列溢出概率与时延违反概率之间的定量关系，从而为将尾部风险嵌入资源调度约束提供理论依据[10]。然而，静态尾部分布建模不足以应对工业无线环境的动态变化，风险约束必须与在线资源决策机制耦合，在随机优化框架下转化为可计算的稳定性条件，并结合智能学习实现多维决策变量的自适应联合优化。

基于上述考虑，本章提出一种基于极值理论辅助强化学习的低时延高能效资源调度机制。首先，引入 EVT 中的 POT 模型对任务队列的尾部统计特性进行精细建模，通过将超阈值队列长度样本拟合为广义帕累托分布（Generalized Pareto Distribution, GPD），并约束其一阶矩和二阶矩，可有效控制极端排队时延的发生概率，从而在统计意义上保障时延的确定性。其次，将 GPD 约束与可靠性约束转化为虚拟队列稳定性约束，利用 Lyapunov 优化理论将长期随机优化问题分解为逐时隙的确定性子问题。最后，针对多维决策变量的联合优化，设计基于 WoLF-PHC 的分布式强化学习算法，实现多用户场景下的实时资源调度。本章的主要贡献概括如下：

（1）针对上行实时业务随机到达引发的队列长尾时延问题，构建基于极值理论的尾部统计建模方法，引入峰值超阈值模型对超阈值队列样本进行广义帕累托分布拟合，通过约束其期望和方差实现对长尾时延的有效控制，为确定性服务质量的概率保障提供理论基础。

（2）将广义帕累托分布约束与队列溢出概率约束转化为虚拟队列稳定性条件，利用 Lyapunov 漂移-惩罚函数将长期随机优化问题分解为逐时隙确定性子问题，实现尾部风险控制与能效优化的协同设计，在降低求解复杂度同时保持理论可控性。

（3）基于最大似然估计在线拟合广义帕累托分布，设计 WoLF-PHC 多域资源联合调度算法实现卸载决策与资源的联合优化。仿真表明，在保障关键任务时延违约概率前提下，所提算法较本地计算和传统 MEC 分配分别降低能耗约 35% 和 22%，抑制长尾时延风险，体现出更优的稳定性与鲁棒性。

1.2 系统模型

本章考虑由单基站与多工业无线终端构成的工业无线网络，如图 3-1 所示。基站侧部署一台 MEC 服务器，为N个工业设备提供计算卸载服务。工业设备集合记为$\mathcal{N} = \{1,2,\dots,N\}$，设备受自身计算能力、能量容量约束，无法独立完成高复杂度、低时延要求的计算任务，需根据业务需求自主选择本地执行或 MEC 卸载。为适配工业业务的离散化处理特性，将系统时间轴离散化为等长时隙，每个

时隙的长度为τ，时隙集合记为$\mathcal{T}=\{0,1,2,\ldots,T\}$。所有任务的生成、决策与执行均以时隙为基本单位。

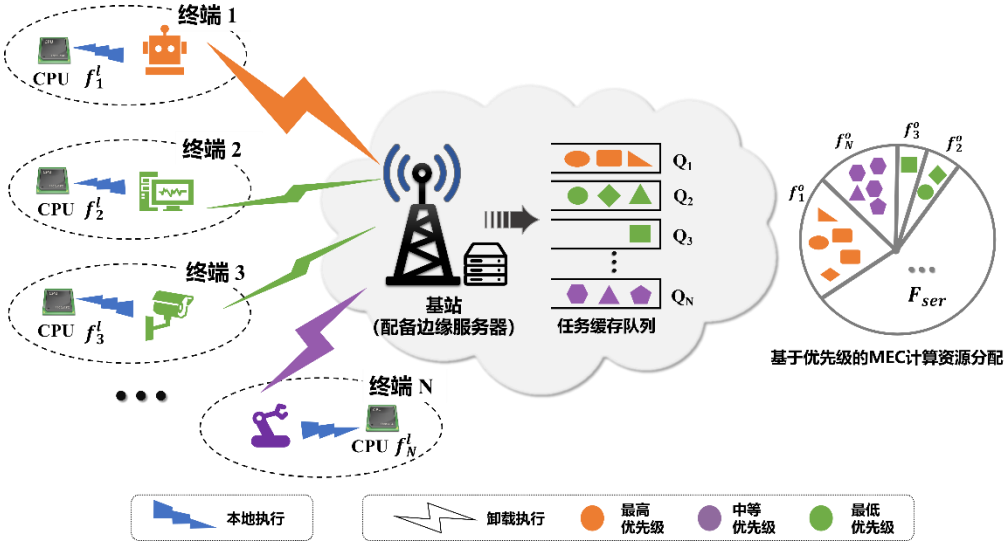


图 3-1　系统模型图

令$A_i(t)$表示设备$i\in\mathcal{N}$在时隙$t\in\mathcal{T}$内产生的计算任务，网络内所有设备的任务集为$\{A_i(t)|i\in\mathcal{N},t\in\mathcal{T}\}$。由于不同工业任务在时延阈值、可靠性要求以及计算复杂度等方面存在显著差异，每个设备可自主选择任务的执行模式，将任务卸载至 MEC 服务器执行或在本地完成计算。

1.2.1　任务模型与优先级策略

工业无线网络中的上行实时业务通常包含周期性的状态采样数据和事件触发的告警信息，任务属性呈现高度异构性。为实现对关键任务的优先保障，必须设计一种能够量化任务紧急程度和服务需求的优先级指标。本章提出一种融合时延约束与可靠性约束的任务优先级计算方法，综合考虑任务的时延阈值与可靠性需求，为每个任务赋予量化的优先级值，其定义如下：

$$Priority=\frac{1}{\mathrm{Time}+\alpha\times\mathrm{Reliability}},\qquad 1.$$

其中，$Time$表示任务的时延阈值t^{max}，该值越小，任务对时延越敏感，优先级越高；α为时延与可靠性之间的权衡参数且满足$0<\alpha\leq 1$，用于调节两个指标的相对重要性。可靠性指标定义为$Reliability=\lambda+\beta\cdot\epsilon$，其中$\lambda$表示最大可容忍的队列溢出概率，$\epsilon$表示最大可容忍的传输误码率，$\beta$为权衡参数且满足$0<\beta\leq 1$，用于调节队列溢出与传输误码在可靠性中的权重。参数α和β可根据具体工业应用场景进行预设，例如在闭环控制中可侧重时延，而在告警上报中可侧重可靠性。该优先级函数实现以下映射关系：时延阈值越小，优先级越高；可容忍违约概率越低，优先级越高。

为简化分析，本章假设同一设备产生的任务具有相同的 QoS 需求（即相同的λ、ϵ和t^{max}），因此共享相同的优先级值。设Pr_i表示设备i在时隙t的任务优先级值。在时隙t卸载的所有任务中，优先级较高的任务将在 MEC 服务器端获得更多的带宽和计算资源。

为简化分析并聚焦于资源调度机制，本章假设同一工业设备产生的所有任务具有相同的 QoS 需求（即相同的λ、ϵ和t^{max}），因此共享相同的优先级值Pr_i。这一假设在设备功能固定的工业场景中是合理的，例如一个温度传感器始终以固定周期上传数据，其 QoS 需求基本不变。在任意时隙t，所有卸载至 MEC 服务器的任务中，优先级值越高的任务将获得更多的无线带宽与 MEC 计算资源，实现关键任务的资源优先保障。

工业应用中，不同终端承担不同功能，其产生的任务在时延阈值与可靠性要求方面存在差异。为统一描述任务特征，将设备i在时隙t产生的任务可表示为三元组：

$$A_i(t) \triangleq \{b_i(t), d_i(t), P_{r,i}\}, \tag{2.}$$

其中，$b_i(t)$表示任务$A_i(t)$的数据量大小，$d_i(t)$表示处理单位比特数据所需的计算资源（以 CPU 周期数计），两者共同决定任务的计算负荷。Pr_i表示任务优先级值，将用于指导后续的资源分配。

定义$\theta_i(t)$表示设备i在时隙t的任务卸载决策：当$\theta_i(t) = 1$时，任务$A_i(t)$被卸载至 MEC 服务器执行；当$\theta_i(t) = 0$时，任务在本地执行。系统在时隙t的卸载决策集合记为$\boldsymbol{\Lambda}(t) = [\theta_1(t), \theta_2(t), \dots, \theta_N(t)]$，其中$\boldsymbol{\Lambda}(t) \in \{0,1\}^N$，全局卸载决策直接决定系统的无线传输负载与 MEC 计算负载分布。

1.2.2 本地计算模型

当任务选择本地计算时，其处理能力受限于工业设备的本地 CPU 性能。为在满足任务时延约束的同时尽可能降低能耗，设备可采用动态电压频率调节（Dynamic Voltage and Frequency Scaling, DVFS）技术动态调整 CPU 工作频率[错误!未找到引用源。]。DVFS 技术通过降低电压和频率来减少功耗，但会相应延长任务执行时间，因此需要在时延与能耗之间进行权衡。设所有设备的本地 CPU 频率为$\boldsymbol{f}^l(t) \triangleq \left[f_1^l(t), f_2^l(t), \dots, f_N^l(t)\right]$，其中各设备的 CPU 频率受硬件上限约束，即$f_i^l(t) \le F_{i,\max}^l, \forall i \in \mathcal{N}$。频率值可根据当前任务量和时延要求在一定范围内连续调节。

对于在本地执行的任务$A_i(t)$，其计算时延主要取决于任务数据量、所需计算资源以及分配的 CPU 频率。具体地，计算时延可表示为：

$$T_i^l = \frac{d_i(t) \cdot b_i(t)}{f_i^l(t)}. \tag{3.}$$

根据 CMOS 电路功耗模型，CPU 的动态功耗与工作频率的三次方成正比[错误!未找到引用源。]，因此本地执行所产生的计算能耗可表示为：

$$E_i^l(t) = \kappa_{mob,i} \cdot [f_i^l(t)]^3 \cdot \tau, \quad 4.$$

其中，$\kappa_{mob,i}$为与设备i的 CPU 硬件特性相关的能耗系数；τ为时隙长度，由于每个时隙内设备可能仅部分时间处于工作状态，此处假设设备在整个时隙内以恒定频率运行，能耗按全时隙计算。实际中可根据占空比进行调整，但为简化模型，本章采用此近似。

从式 4 可见，能耗随频率呈三次方增长，因此适当降低频率可在时延允许的范围内大幅节约能量。例如，若时延约束较宽松，设备可选择较低频率运行，从而在牺牲一定时延性能的情况下显著降低能耗。这种灵活性为后续的资源优化提供了空间。

1.2.3　卸载计算模型

当任务选择卸载至 MEC 服务器执行时，其时延和能耗由无线传输与边缘计算两个阶段共同决定。相较于本地执行，卸载模式能够借助服务器的强大算力缩短计算时延，但需额外考虑上行传输开销与资源竞争的影响。

为实现高优先级任务的差异化保障，系统采用优先级加权的带宽分配策略，根据任务优先级动态分配无线带宽资源。设系统总无线带宽为BW（单位：Hz），在时隙t内将带宽按比例$\gamma_i(t)$分配给卸载设备i，则其占用带宽为$\gamma_i(t)BW$，并满足$\sum_{i\in\mathcal{N}} \gamma_i(t) = 1$且$0 \le \gamma_i(t) \le 1$。考虑任务的差异化 QoS 需求，本章采用基于优先级加权的带宽分配规则：

$$\gamma_i(t) = \frac{\theta_i(t) \cdot \mathrm{Pr}_i}{\sum_{j\in\mathcal{N}} \theta_j(t) \cdot \mathrm{Pr}_j}, i \in \mathcal{N}, t \in \mathcal{T}. \quad 5.$$

由此，只有发生卸载的终端参与带宽归一化分配，并且优先级越高的任务获得更高的带宽份额。该分配机制确保高优先级任务在资源竞争中占据优势，从而满足其严苛的时延可靠性要求。

无线信道的随机特性直接影响任务卸载的传输速率与可靠性，传统香农公式仅适用于长数据包传输场景，难以准确刻画工业实时业务的短包传输特性。因此，本章基于有限块长编码理论（Finite Block-length Coding, FBL）构建传输速率模型，该理论能够精准描述短数据包传输场景下的速率-可靠性权衡关系，更贴合工业实际需求。设备i的上行传输速率采用有限块长近似表达式[错误!未找到引用源。]建模为

$$v_i(t) = \gamma_i(t) \cdot BW \cdot (\log_2(1 + \delta_i(t)) - \psi_i(t)), \quad 6.$$

其中，$\delta_i(t)$为时隙t内接收上行信号的信噪比（Signal-to-Noise Ratio, SNR），ψ为可靠性因子，用于量化传输误码率约束对速率的影响，定义为$\psi_i(t) = \sqrt{V_\tau / n_i(t)} \cdot Q^{-1}(\epsilon_i)$。式中$V_\tau$表示信道色散系数（通常可设为 1 [错误!未找到引用源。]），用于衡量信道随机波动程度；Q^{-1}表示高斯 Q 函数的逆函数；n为块长；ϵ_i表示任务$A_i(t)$的最大可容忍传输误码率。在工业关键任务场景中，可靠性因子$\psi_i(t)$通常设置为10^{-5}量级，确保传输可靠性满足严苛要求。

接收信噪比$\delta_i(t)$由发射功率、信道衰落与噪声功率共同决定，是刻画信道质量的核心指标，定义为：

$$\delta_i(t) = \frac{g_0 \cdot p_{tx,i}(t)}{N_0 \cdot \gamma_i(t) \cdot BW}, \tag{7.}$$

其中，$p_{tx,i}(t)$表示设备i在时隙t用于任务卸载的发射功率，受限于最大值为$p_{tx,i}^{max}(t)$；N_0表示噪声功率谱密度；g_0表示归一化到距离的信道衰落系数，综合考虑了路径损耗与阴影衰落。

基于上述速率，任务$A_i(t)$的上行传输时延为：

$$t_{up,i}(t) = \frac{b_i(t)}{v_i(t)}. \tag{8.}$$

由于 MEC 服务器可能同时处理多个设备的卸载任务，任务数据到达 MEC 服务器后，需在队列中等待并接受计算服务。任务在服务器端的计算时延为t_{exe}，等待时延为t_{wait}，。计算时延取决于任务的计算负荷与分配的计算资源：

$$t_{exe,i}(t) = \frac{d_i(t) b_i(t)}{f_i^o(t)}, \tag{9.}$$

其中，$f_i^o(t)$为 MEC 服务器分配给任务$A_i(t)$的计算资源，该计算资源的分配同样遵循优先级原则，将在下一节详细描述。等待时延等于队列中排在任务$A_i(t)$之前的所有任务的执行时间之和，即

$$t_{\mathrm{wait},i}(t) = \sum\nolimits_{k \in Q_i^{\mathrm{prve}}(t)} t_{\mathrm{exe},k}(t), \tag{10.}$$

其中，$Q_i^{\mathrm{prve}}(t)$为 MEC 任务队列中排在任务$A_i(t)$之前的所有任务集合。在实际系统中，等待时延待取决于队列的瞬时状态，难以精确解析，但可通过队列演化方程进行动态跟踪。

卸载计算的最后一步是将 MEC 服务器把处理后的任务返回给设备i，但通常情况下下载数据速率一般都很高，且任务处理结果的数据大小比输入数据小得多

错误!未找到引用源。，因此本章不考虑了这一步骤的时延及能耗。综上，任务卸载执行的总时延为传输时延、等待时延与计算时延之和，即

$$T_i^o(t) = t_{up,i}(t) + t_{wait,i}(t) + t_{exe,i}(t). \tag{11}$$

总时延需严格满足任务的时延阈值约束$T_i^o(t) \le t_{i,\max}$，否则将引发 QoS 违约。

任务卸载的总能耗包括设备端传输能耗与 MEC 服务器计算能耗两部分，具体表达式为：

$$E_i^o(t) = E_{tx,i}(t) + E_{ser,i}(t), \tag{12}$$

其中，设备端传输能耗由发射功率与传输时间决定，计算公式为$E_{tx,i}(t) = p_{tx,i}(t) \cdot t_{up,i}(t)$，MEC 服务器的计算能耗遵循 CMOS 电路能耗特性，与计算频率的平方呈正相关，计算公式为$E_{ser,i}(t) = \kappa_{ser} \cdot [f_i^o(t)]^2 \cdot d_i(t) \cdot b_i(t)$，$\kappa_{ser}$为 MEC 服务器 CPU 的能耗系数，取决于服务器 CPU 的架构与制程工艺。综上，任务卸载的总能耗为

$$E_i^o(t) = E_{up,i}(t) + E_{ser,i}(t). \tag{13}$$

1.2.4　任务队列模型

由于 MEC 服务器计算资源有限，当多个设备同时卸载任务时，不可避免地产生资源竞争与排队现象。MEC 服务器为每个工业设备维护一个独立的任务队列用于缓存已接收但尚未执行的卸载任务。

在时隙t内，到达任务队列的数据量为$\theta_i(t)\, b_i(t)$，仅卸载任务入队，本地执行任务不进入 MEC 队列。在每个时隙，MEC 服务器根据各设备的优先级和队列状态动态分配计算资源。具体地，分配给设备i在的计算资源为：

$$f_i^o(t) = F_{ser} \cdot \frac{Pr_i \cdot I_{\{Q_i(t)>0\}}}{\sum_{j\in\mathcal{N}} Pr_j \cdot I_{\{Q_i(t)>0\}}}, \tag{14}$$

其中，F_{ser}为 MEC 服务器在时隙t的总计算资源（单位：cycle/s），$I\{\cdot\}$为指示函数，用于排除空队列对资源分配的影响。由此可见，当队列非空时，优先级越高的队列获得更高的服务速率。

设$Q_i(t)$表示设备i在时隙t开始时的队列长度。在每个时隙内，新到达的任务数据$\theta_i(t)\, b_i(t)$加入队列，同时服务器执行完成的任务数据$\tau \cdot f_i^o(t)/L_i$从队列中移除，其中L_i为处理密度（单位比特所需的 CPU 周期数）。因此，队列的演化遵循以下动态方程：

$$Q_i(t) = \max\left\{Q_i(t-1) + \theta_i(t) \cdot b_i(t) - \frac{\tau \cdot f_i^o(t)}{L_i}, 0\right\}. \tag{15}$$

该方程反映了队列积压的时变特性，当任务到达率超过服务率时，队列长度增加；反之则减少。队列长度的稳定性是系统能够持续提供可靠服务的前提。

1.3 优化问题建模

1.3.1 系统优化目标

基于前述系统模型，本节旨在建立上行实时业务随机到达场景下的资源调度优化问题。问题的核心目标是在满足端到端时延阈值与可靠性约束的前提下，最小化系统的长期平均能耗。根据前文所述工业无线网络的总能耗是所有设备本地执行能耗与卸载执行能耗的总和，考虑到工业业务的随机到达特性与信道的随机波动特性，瞬时能耗无法反映系统的真实运行性能，因此本章的优化目标为：

$$E_{all} = \lim_{T\to\infty} \frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E}\left[\sum_{i\in\mathbf{N}} \left((1-\theta_i)E_i^l(t) + \theta_i E_i^o(t)\right)\right]. \tag{16.}$$

1.3.2 队列均值稳定性和队列溢出概率约束

首先，队列稳定性是系统正常运行的前提，要求队列长度的长期时间平均值有限，避免任务无限积压导致服务崩溃。

$$\lim_{T\to\infty} \frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E}[Q_i] < \infty, i \in \mathcal{N}. \tag{17.}$$

其次，服务器缓存空间有限，设定队列阈值q_0与最大可容忍溢出概率λ_i，要求

$$\Pr(Q_i(t) \geq q_0) \leq \lambda_i, i \in \mathcal{N}, t \in \mathcal{T}. \tag{18.}$$

式 17 保证队列不会无限增长，18 则将队列长度超过阈值的概率控制在可接受范围内。然而，仅靠 18 仍存在不足：即便违约概率被压低，越界一旦发生，队列可能出现极端峰值，进而造成更严重的长尾时延。因此，需要进一步对违约后的尾部行为建模并约束。

1.3.3 基于极值理论的队列尾部风险约束

在随机突发到达条件下，队列过程可能出现显著的长尾行为，对于工业闭环控制、设备远程操控等关键任务而言，这种极端时延事件即使发生概率很低，也可能导致控制指令延迟、生产流程中断等严重后果。因此本章引入极值理论，刻画任务队列的尾部统计特性。与中心极限定理关注样本均值不同，极值理论聚焦于样本分布的尾部形态，能够对“超越阈值”的极端事件进行精确建模。在众多极值理论方法中，峰值超阈值（Peaks-Over-Threshold, POT）模型因其对数据的有效利用而成为工程应用中最广泛采用的方法。

POT 模型的理论基础源于 Pickands-Balkema-de Haan 定理[错误!未找到引用源。]。该定理指出：对于一个随机变量Q，其累积分布函数为$F_Q(q)$。随着阈值d充分接近于F_Q的支撑集上界（即$d \to \sup\{q: F_Q(q) < 1\}$），超阈值样本$\mathbf{X}|_{Q>d} = Q - d > 0$的条件分布可以用广义帕累托分布（Generalized Pareto Distribution, GPD）进行近似。具体地，存在尺度参数$\sigma > 0$和形状参数$\xi \in \mathbb{R}$，使得：$\Pr(Q - d > x \mid Q > d) \approx 1 - G(x;\sigma,\xi)$，其中广义帕累托分布$G(x;\sigma,\xi)$的定义域是当$\xi \geq 0$时，$x \geq 0$；当$\xi < 0$时，$0 \leq x \leq -\sigma/\xi$，GPD 的表达式为：

$$G(x;\sigma,\xi) = \begin{cases} 1 - \left(1 + \frac{\xi_x}{\sigma}\right)^{-1/\xi}, & \xi \neq 0 \\ 1 - \exp\left(-\frac{x}{\sigma}\right), & \xi = 0 \end{cases}. \tag{19.}$$

GPD 的均值与方差分别表示为

$$\mathbb{E}[\mathbf{X}] = \frac{\sigma}{1-\xi}, \ \mathrm{Var}(\mathbf{X}) = \frac{\sigma^2}{(1-\xi)^2(1-2\xi)}. \tag{20.}$$

形状参数ξ决定了尾部的厚薄程度：$\xi > 0$对应厚尾分布（如帕累托分布），$\xi = 0$对应指数尾部（如指数分布），$\xi < 0$对应薄尾分布（如均匀分布）。对于工业无线网络中由随机突发到达引发的队列积压，通常表现出厚尾特征，因此$\xi > 0$是常见情形。

当形状参数满足$\xi < 1/2$时，GPD 的一阶矩和二阶矩存在。由 20 可得到二阶原点矩为

$$\mathbb{E}[\mathbf{X}^2] = \mathrm{Var}(\mathbf{X}) + (\mathbb{E}[\mathbf{X}])^2 = \frac{2\sigma^2}{(1-\xi)(1-2\xi)}. \tag{21.}$$

将上述极值理论应用于本章的任务队列分析。定义超阈值随机变量$Z_i(t) \triangleq Q_i(t) - q_0$，对应超过阈值$q_0$的队列长度样本集合（即极值样本集）为

$$\mathbb{Z}_i = \{Z_i(t) \mid Q_i(t) > q_0\}. \tag{22.}$$

当阈值q_0足够大时，极值样本$Z_i(t)$将渐近服从广义帕累托分布$G_{\sigma_i,\xi_i}(Z_i)$。基于上述分析，本章通过限制超阈值样本的长期一阶与二阶矩来约束队列长度的尾部特性。具体地，定义队列尾部风险约束如下：

$$\lim_{T\to\infty} \frac{\sum_{t=1}^{T} (Q_i(t) - q_0) \cdot I_{\{Q_i(t)>q_0\}}}{\sum_{t=1}^{T} I_{\{Q_i(t)>q_0\}}} \leq \frac{\sigma_i}{1-\xi_i}, i \in \mathcal{N} \tag{23.}$$

$$\lim_{T\to\infty} \frac{\sum_{t=1}^{T} (Q_i(t) - q_0)^2 \cdot I_{\{Q_i(t)>q_0\}}}{\sum_{t=1}^{T} I_{\{Q_i(t)>q_0\}}} \leq \frac{2\sigma_i^2}{(1-\xi_i)(1-2\xi_i)}, i \in \mathcal{N} \tag{24.}$$

该约束的含义是在违约事件发生的条件下，队列峰值的平均大小与波动幅度被限制在可控范围内，从而有效抑制长尾时延风险。参数集$d_i = [\sigma_i, \xi_i]$并非预先给定的常数，而是需要通过历史数据在线估计的 GPD 参数。本章将在后续算法中采用最大似然估计（Maximum Likelihood Estimation, MLE）对 GPD 参数进行实时更新[错误!未找到引用源。]。

1.3.4 优化问题的提出

综合上述分析，本章将优化问题建模为带队列均值稳定性、队列溢出概率约束和队列尾部风险约束的长期随机优化问题，表述为：

$$(\mathbf{P}): \min_{\boldsymbol{\Lambda}(t), f^l(t), p_{tx}(t)} E_{all}. \tag{25.}$$

$$\begin{aligned}
\text{s.t. } & \text{C1: } \theta_i(t) \in \{0,1\}, \\
& \text{C2: } 0 < f_i^l(t) \le F_{i,max}^l, \\
& \text{C3: } 0 < p_{tx,i}(t) \le p_{tx,i}^{max}, \\
& \text{C4: } \sum_{i \in \mathcal{N}} \gamma_i(t) = 1, 0 \le \gamma_i(t) \le 1, \\
& \text{C5: } (1-\theta_i(t)) T_i^l(t) + \theta_i(t) T_i^o(t) \le t_{i,max}, \\
& \text{C6: } \mathbb{E}[Q_i] < \infty, \Pr(Q_i(t) \ge 0) \le \lambda_0, \\
& \text{C7: } \lim_{T \to \infty} \frac{\sum_{t=1}^{T} (Q_i(t) - q_0) \cdot I_{\{Q_i(t) > q_0\}}}{\sum_{t=1}^{T} I_{\{Q_i(t) > q_0\}}} \le \frac{\sigma_i}{1-\xi_i}, \\
& \text{C8: } \lim_{T \to \infty} \frac{\sum_{t=1}^{T} (Q_i(t) - q_0)^2 \cdot I_{\{Q_i(t) > q_0\}}}{\sum_{t=1}^{T} I_{\{Q_i(t) > q_0\}}} \le \frac{2\sigma_i^2}{(1-\xi_i)(1-2\xi_i)}.
\end{aligned} \tag{26.}$$

约束$C1$表示卸载决策为二元变量，表示任务只能选择本地执行或完全卸载。；约束$C2$和$C3$分别限定本地 CPU 频率和发射功率的可行范围；约束$C4$保证带宽资源分配的完备性；约束$C5$确保任务执行时延不超过其时延阈值要求；约束$C6$保证队列均值稳定性和队列溢出概率约束；约束$C7$和$C8$为基于极值理论的尾部统计约束，用于控制超阈值队列长度的分布特性，从而保障关键任务在极端情况下的时延性能。上述问题**P**属于带随机约束的长期随机优化问题，直接求解该问题难度极大。首先，决策变量包括卸载选择（二元变量）、本地 CPU 频率（连续变量）和发射功率（连续变量），属于混合整数非线性规划问题；其次，目标函数和约束涉及随机过程的长期统计特性，难以直接求解；最后，尾部统计约束中的 GPD 参数未知且需要在线估计。为此，下一节将引入李雅普诺夫（Lyapunov）优化框架将问题转化为逐时隙的确定性优化，并结合极值参数估计与强化学习算法实现高效求解。

1.4 基于李雅普诺夫优化与强化学习的任务卸载与资源分配算法

本节针对 3.2 节建立的优化问题(P)，提出基于李雅普诺夫优化与强化学习相结合的任务卸载与资源分配求解方案。首先引入李雅普诺夫优化框架[错误!未找到引用源。]，将队列稳定性、GPD 尾部特性等复杂约束转化为虚拟队列稳定性约束，将长期随机优化问题分解为逐时隙的确定性上界最小化问题。随后，采用最大似然估计结合随机方差缩减梯度下降方法，实现 GPD 参数的在线精准估计。最后，采用基于 WoLF-PHC 的分布式强化学习算法，完成卸载决策、本地 CPU 频率、发射功率的多维联合优化。

1.4.1　基于李雅普诺夫优化的约束转化与问题简化

1.4.1.1　虚拟队列构建与约束转化

为了将原问题中的可靠性约束$(C6)$与 GPD 尾部特性约束$(C7\text{-}C8)$转化为可处理的队列稳定性约束，本节构建了三类虚拟队列，分别对应不同约束的量化表征。

根据马尔可夫不等式，队列溢出概率约束$\Pr(Q_i(t) \geq q_0) \leq \lambda_0$可转化为队列长度的期望约束：

$$\mathbb{E}[Q_i(t)] \leq \lambda_0 \cdot q_0. \tag{27}$$

为保障该期望约束成立，构建队列溢出概率约束虚拟队列$Q_i^X(t)$，其更新方程为：

$$Q_i^X(t+1) = \max\{Q_i^X(t) + Q_i(t+1) - \lambda_0 q_0, 0\}, \tag{28}$$

其中，$Q_i(t+1)$为实际任务队列长度，$\lambda_0 q_0$为期望约束阈值。当虚拟队列$Q_i^X(t)$稳定时（即$\mathbb{E}[Q_i^X(t)] < \infty$），可确保实际队列的期望约束 27 成立，进而满足队列溢出概率约束$(C6)$。

GPD 一阶矩约束要求超阈值队列长度样本的长期时间平均期望不超过$\sigma_i/(1-\xi_i)$，构建虚拟队列$Q_i^Y(t)$刻画该约束的满足程度，更新方程为：

$$Q_i^Y(t+1) = \max\left\{Q_i^Y(t) + \left(Q_i(t+1) - q_0 - \frac{\sigma_i}{1-\xi_i}\right) \cdot I_{\{Q_i(t)>q_0\}}, 0\right\}. \tag{29}$$

GPD 二阶矩约束要求超阈值队列长度样本的长期时间平均方差不超过$2\sigma_i^2/[(1-\xi_i)(1-2\xi_i)]$，构建虚拟队列$Q_i^Z(t)$，更新方程为：

$$Q_i^Z(t+1) = \max\left\{Q_i^Z(t) + \left[(Q_i(t+1) - q_0)^2 - \frac{2\sigma_i^2}{(1-\xi_i)(1-2\xi_i)}\right] \cdot I_{\{Q_i(t)>q_0\}}, 0\right\}. \tag{30}$$

当$Q_i^Y(t)$和$Q_i^Z(t)$稳定时，可确保一阶矩约束$(C7)$和二阶矩约束$(C8)$成立。

1.4.1.2　Lyapunov 漂移-惩罚函数设计

为分析实际队列与虚拟队列的联合稳定性，定义 Lyapunov 函数$L\big(\Phi(t)\big)$，其中$\Phi(t)=\{Q_i(t),Q_i^X(t),Q_i^Y(t),Q_i^Z(t)|i\in\mathcal{N}\}$为系统联合状态向量：

$$L\big(\Phi(t)\big)=\frac{1}{2}\sum_{i\in\mathcal{N}}\left[Q_i^2(t)+\left(Q_i^X(t)\right)^2+\left(Q_i^Y(t)\right)^2+\left(Q_i^Z(t)\right)^2\right]. \qquad 31.$$

定义 Lyapunov 漂移函数定义为相邻时隙间 Lyapunov 函数的变化量：

$$\Delta L_t=L\big(\Phi(t+1)\big)-L\big(\Phi(t)\big). \qquad 32.$$

通过控制 Lyapunov 漂移的上界，可保证实际队列与虚拟队列的稳定性。然而，单纯追求队列稳定可能导致系统能耗过高。为在保障队列稳定的同时实现能耗最小化，本节定义漂移-惩罚函数：

$$\Delta L_t+V\cdot\mathbb{E}\{E_{all}|\Phi(t)\}. \qquad 33.$$

其中，$V>0$为控制参数，用于平衡队列稳定性与能耗优化之间的权衡。V值越大，算法越倾向于能耗最小化；V值越小，算法越倾向于队列稳定性保障。

1.4.1.3　漂移-惩罚函数上界推导与问题转化

通过将各队列的更新方程代入漂移-惩罚函数，并进行放缩处理，可推导出其上界。

定理 3.1 漂移-惩罚函数的上界如下不等式：

$$\begin{aligned}\Delta L_t+V\cdot\mathbb{E}\{E_{\text{all}}\mid\Phi(t)\}\le\Theta\\ =\sum_{i\in\mathcal{N}}&[\Delta_i+\theta_i(t)b_i(t)\{(1+Q_i^X(t)-\lambda_0q_0)Q_i(t)-\lambda_0Q_i^X(t)\\ &+[2(Q_i(t)-q_0)\cdot(Q_i^Z(t)+(Q_i(t)-q_0)^2-M_2)\\ &+Q_i(t)+Q_i^Y(t)-M_1]\cdot I_{\{Q_i(t)>q_0\}}\}]+\Delta_0+V\cdot\mathbb{E}\{E_{\text{all}}\mid\Phi(t)\}\end{aligned} \qquad .34.$$

其中，$M_1=q_0+\frac{\sigma_i}{1-\xi_i}$和$M_2=\frac{2\sigma_i^2}{(1-\xi_i)(1-2\xi_i)}$分别为队列长度尾部分布的一阶矩和二阶矩边界；$\Delta_i$表示与控制变量无关的项集合；$\Delta_0$为常数项。

证明：首先，考虑 Lyapunov 漂移函数，漂移-惩罚函数可展开为：

$$\begin{aligned}&\Delta L_t+\mathrm{V}\cdot\mathbb{E}\{E_{all}\mid\Phi(\mathrm{t})\}\\ &=\frac{1}{2}\sum_{i\in N}\left[Q_i^2(t+1)-Q_i^2(t)+\left(Q_i^X(t+1)\right)^2-\left(Q_i^X(t)\right)^2+\left(Q_i^Y(t+1)\right)^2-\left(Q_i^Y(t)\right)^2\right.\\ &\left.+\left(Q_i^Z(t+1)\right)^2-\left(Q_i^Z(t)\right)^2\right]+\mathrm{V}\cdot\mathbb{E}\{E_{all}\mid\Phi(\mathrm{t})\}\end{aligned} \qquad .35.$$

利用不等式$(\max q+a-b,0)^2\le q^2+(a-b)^2+2q(a-b)$，结合任务队列更新方程和虚拟队列更新方程，可推导各项的上界如下：

$$\frac{Q_i^2(t+1)-Q_i^2(t)}{2} \le \left[\frac{(\theta_i(t)b_i(t)-\tau\cdot f_i^o(t)/L_i)^2}{2}\right]_{\#a1} - [Q_i(t)\cdot(\tau\cdot f_i^o(t)/L_i)]_{\#b1} + [Q_i(t)\cdot$$

$$\begin{aligned}\frac{\left(Q_i^X(t+1)\right)^2-\left(Q_i^X(t)\right)^2}{2} \le& \left[\frac{(Q_i^2(t)+(\theta_i(t)b_i(t)-\tau\cdot f_i^o(t)/L_i)^2+\lambda_0^2q_0^2)}{2}\right]_{\#a2}\\ &+\left[Q_i^X(t)Q_i(t)-\left(Q_i^X(t)+Q_i(t)\right)(\tau\cdot f_i^o(t)/L_i+\lambda_0q_0)+\lambda_0q_0\cdot(\tau\cdot f_i^o(t)\right.\\ &+[\theta_i(t)b_i(t)(Q_i^X(t)+Q_i(t)-\lambda_0q_0)]\end{aligned}$$

$$\begin{aligned}\frac{\left(Q_i^Y(t+1)\right)^2-\left(Q_i^Y(t)\right)^2}{2} \le& \left\{\left[\frac{Q_i^2(t)+(\theta_i(t)b_i(t)-\tau\cdot f_i^o(t)/L_i)^2+M_1^2}{2}\right]_{\#a3}\right.\\ &+\left[Q_i^Y(t)Q_i(t)-\left(Q_i^Y(t)+Q_i(t)\right)(\tau\cdot f_i^o(t)/L_i+M_1)+M_1\cdot(\tau\cdot f_i^o(t)/L_i)\right.\\ &\left.+[\theta_i(t)b_i(t)(Q_i^Y(t)+Q_i(t)-M_1)]\right\}\cdot I_{\{Q_i(t)>q_0\}}\end{aligned}$$

$$\begin{aligned}\frac{\left(Q_i^Z(t+1)\right)^2-\left(Q_i^Z(t)\right)^2}{2} \le& \left[\left[\frac{((Q_i(t)-q_0)^2+(\theta_i(t)b_i(t)-\tau\cdot f_i^o(t)/L_i)^2-M_2)^2}{2}\right.\right.\\ &+2\left((Q_i(t)-q_0)^2+(Q_i(t)-q_0)(\theta_i(t)b_i(t)-\tau\cdot f_i^o(t)/L_i)\right)(\theta_i(t)b_i(t)-\\ &+((Q_i(t)-q_0)^2+(\theta_i(t)b_i(t)-\tau\cdot f_i^o(t)/L_i)^2)\cdot Q_i^Z(t)]_{\#a4}^2\\ &-\left[2(Q_i(t)-q_0)(\tau\cdot f_i^o(t)/L_i)\cdot\left((Q_i(t)-q_0)^2-M_2+Q_i^Z(t)\right)\right]_{\#b4}\\ &\left.+\left[2\theta_i(t)b_i(t)(Q_i(t)-q_0)\left((Q_i(t)-q_0)^2-M_2+Q_i^Z(t)\right)\right]\right]\cdot I_{\{Q_i(t)>q_0\}}\end{aligned}$$

其中，$M_1 = q_0 + \frac{\sigma_i}{1-\xi_i}$，$M_2 = \frac{2\sigma_i^2}{(1-\xi_i)(1-2\xi_i)}$。考虑到$\#a1$、$\#a2$、$\#a3$和$\#a4$项均为二次项，可将其求和并以有界常数$\Delta_0$进行上界替换：

$$\begin{aligned}\Delta_0 \ge& \frac{(\theta_i(t)b_i(t)-\tau\cdot f_i^o(t)/L_i)^2}{2}+\frac{(Q_i^2(t)+(\theta_i(t)b_i(t)-\tau\cdot f_i^o(t)/L_i)^2+\lambda_0^2q_0^2)}{2}\\ &+\left\{\frac{Q_i^2(t)+(\theta_i(t)b_i(t)-\tau\cdot f_i^o(t)/L_i)^2+M_1^2}{2}+\frac{((Q_i(t)-q_0)^2+(\theta_i(t)b_i(t)-\tau\cdot f_i^o(t)/L_i)^2-M\right.\\ &+2\left((Q_i(t)-q_0)^2+(Q_i(t)-q_0)(\theta_i(t)b_i(t)-\tau\cdot f_i^o(t)/L_i)\right)(\theta_i(t)b_i(t)-\tau\cdot f_i^o(t)/L_i)^2\\ &+((Q_i(t)-q_0)^2+(\theta_i(t)b_i(t)-\tau\cdot f_i^o(t)/L_i)^2)\cdot Q_i^Z(t)\}\cdot I_{\{Q_i(t)>q_0\}}.\end{aligned}$$

将$\#b1$、$\#b2$、$\#b3$和$\#b4$项视为与控制变量相关的独立项，记为Δ_i：

$$\begin{aligned}\Delta_i =& Q_i^x(t)Q_i(t)-\left(Q_i^x(t)+Q_i(t)\right)(\tau\cdot f_i^o(t)/L_i+\lambda_0q_0)\\ &+\lambda_0q_0(\tau\cdot f_i^o(t)/L_i)-Q_i(t)(\tau\cdot f_i^o(t)/L_i)\\ &+Q_i^Y(t)Q_i(t)-\left(Q_i^Y(t)+Q_i(t)\right)(\tau\cdot f_i^o(t)/L_i+M_1)\\ &+M_1(\tau\cdot f_i^o(t)/L_i)+2(Q_i(t)-q_0)(\tau\cdot f_i^o(t)/L_i)\\ &\cdot\left((Q_i(t)-q_0)^2-M_2+Q_i^Z(t)\right)\end{aligned} \quad . \qquad 38.$$

综合上述推导结果，将各项代入漂移-惩罚函数展开式，即可得到定理 3-1 中漂移-惩罚函数上界Θ的表达式。证毕。 ■

根据 Lyapunov 优化理论，通过最小化漂移-惩罚函数的上界Θ，可同时实现队列稳定性保障与能耗最小化。因此，原问题$(\mathbf{P})$转化为如下上界最小化问题$(\boldsymbol{P}\text{-}1)$：

$$(\mathbf{P}-\mathbf{1}): \min_{\boldsymbol{\Lambda}(t), f^{l}(t), p_{tx}(t)} \Theta(t) \qquad \text{s.t.} C1-C5. \tag{39.}$$

转化后的问题$(\boldsymbol{P}\text{-}1)$在每个时隙独立求解，约束条件仅保留原问题中的核心可行域约束$(C1-C5)$，求解难度显著降低。但由于M_1、M_2依赖 GPD 参数σ_i、ξ_i，需先完成 GPD 参数的精准估计，才能进一步求解该问题。

1.4.2 基于最大似然估计的 GPD 参数求解

GPD 约束的有效实施依赖于参数$\boldsymbol{d_i}=[\sigma_i,\xi_i]$的准确估计。本节采用最大似然估计方法，结合在线采样策略对 GPD 参数进行动态估计[错误!未找到引用源。]。

1.4.2.1 超阈值队列长度样本采集

为确保样本的独立性与代表性，采用滑动窗口采样机制采集超阈值队列长度样本。定义滑动窗口长度为w（时隙数），窗口集合为$\Gamma_k=(k-1)w,(k-1)w+1,\dots,kw-1(k\in\mathbb{N})$。在时刻$t$，对截至$\lfloor t/\mathrm{w}\rfloor$个窗口分别取最大队列长度位置：

$$t_k^*=\arg\max_{\tau\in\Gamma_k} Q_i(\tau), k\in\{1,\dots,\lfloor t/w\rfloor\}. \tag{40.}$$

因此，设备i在时隙t的超阈值样本集为：

$$S_i(t)=\left\{Z=Q_i(t_k^*)-q_0 \;\middle|\; Q_i(t_k^*)>q_0, t_k^*=\arg\max_{\tau\in\Gamma_k} Q_i(\tau), k\in\{1,\dots,\lfloor t/w\rfloor\}\right\}, \tag{41.}$$

其中，t_k^*为窗口Γ_k内队列长度最大的时隙，$Z=Q_i(t_k^*)-q_0$为超阈值样本，$s_i(t)=|S_i(t)|$为样本量。图 3-2 展示了队列长度超阈值采样过程的示意图。

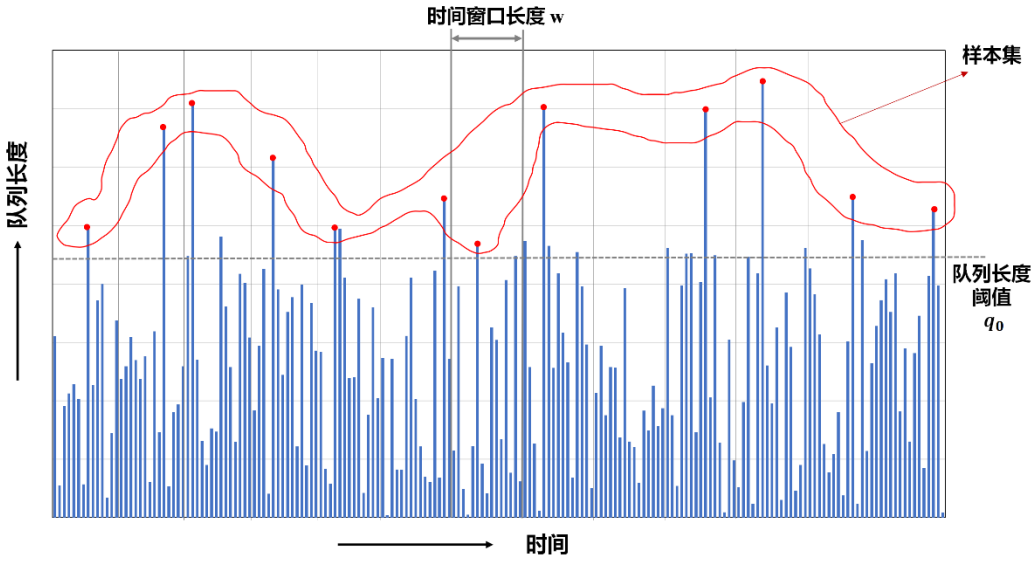


图 3-2 队列长度超阈值采样示意图

1.4.2.2　基于最大似然估计的 GPD 参数估计模型

最大似然估计（MLE）是参数估计的经典方法，其核心思想是找到使样本集$S_i(t)$出现概率最大的参数组合$d_i = [\sigma_i, \xi_i]$。基于采集的样本集合$S_i(t)$，GPD 参数估计问题可通过最小化负对数似然得到，形式化为如下优化问题：

$$\min_{d_i \in d} f^{d_i}(S_i) = -\frac{1}{|S_i|}\sum_{Z \in S_i} \log G^{d_i}(Z), \tag{42.}$$

其中，$\mathcal{D}$为参数可行域：

$$\mathcal{D} = \{[\sigma_i, \xi_i] \in R^2 \mid \sigma_i > 0, \xi_i < 1, 1 + \xi_i Z/\sigma_i \geq 0, \forall Z \in S_i(t)\}. \tag{43.}$$

对负对数似然函数分别求σ_i和ξ_i的偏导数，得到梯度表达式：

$$\nabla_{\mathbf{d}_i} f^{\mathbf{d}_i}(Z) = \begin{bmatrix} \dfrac{\partial f^{\mathbf{d}_i}(Z)}{\partial \sigma_i} \\ \dfrac{\partial f^{\mathbf{d}_i}(Z)}{\partial \xi_i} \end{bmatrix} = \begin{bmatrix} \dfrac{1}{\sigma_i}\left(\dfrac{1+\frac{1}{\xi_i}}{1+\frac{\xi_i Z}{\sigma_i}} - \dfrac{1}{\xi_i}\right) \\ \dfrac{\left(1+\frac{1}{\xi_i}\right)\left(2+\frac{\xi_i Z}{\sigma_i}\right)}{1+\frac{\xi_i Z}{\sigma_i}} - \dfrac{\ln\left(1+\frac{\xi_i Z}{\sigma_i}\right)}{\xi_i^2} \end{bmatrix}. \tag{44.}$$

1.4.2.3　基于随机方差缩减梯度下降的参数优化求解

理论上令式 44 为零可得到估计值，但在线场景下直接求解往往收敛慢且计算代价高。为提高求解效率，本节采用随机方差缩减梯度下降（Stochastic Variance Reduced Gradient Descent, SVRGD）技术，加速参数收敛并提升估计精度。设步长为ℓ，在第n次迭代时，对样本Z的更新为：

$$\begin{cases} y = d_i(n) - \ell\left[\nabla_{d_i} f^{d_i(n)}(Z) - \nabla_{d_i} f^{\tilde{d}_i(n)}(Z) + \zeta_i(n)\right], \\ d_i(n) = \arg \min_{d_i \in d} \parallel y - d_i \parallel. \end{cases} \quad 45.$$

其中，$\tilde{d}_i(n) = \frac{1}{n-1}\sum_{n'=1}^{n-1} d_i(n')$为前$n-1$次迭代的参数平均值，$\zeta_i(n) = \frac{1}{|S_i|}\sum_{Z \in S_i} \nabla_{d_i} f^{\tilde{d}_i(n)}(Z)$为梯度估计量。

通过窗口化采样与 SVRGD-MLE 的结合，系统能够随时间滚动更新d_i，从而实时得到M_1、M_2。该 GPD 参数在线估计算法的具体流程如算法 3-1 所示。

算法3-1：GPD参数在线估计算法

输入： 初始梯度估计$\zeta_i(0)$，初始参数集$d_i(0)$，预设步长ℓ

输出： GPD参数估计$d_i(t)$

1. for $t = 1, 2, \ldots$ do
2. MEC 服务器中设备i对应的任务队列上传队列信息$Q_i(t)$；
3. 从队列信息中提取新增样本集合$\widehat{S}_i(t) = S_i(t) \backslash S_i(t-1)$；
4. 初始化$\zeta_i(t) = 0$，$d_i(t) = d_i(t-1)$；
5. for $n = 1,2, \ldots, |S_i(t)|$ do
6. 根据式44计算$\zeta_i(t)$和$d_i(t)$；
7. 根据$d_i(t)$获取GPD分布特征参数，返回最优参数集$d_i(t)$；

1.4.3 基于 WoLF-PHC 的多智能体联合优化算法

尽管通过 Lyapunov 优化将原问题转化为逐时隙的确定性问题，但问题($\boldsymbol{P}$-1)仍涉及卸载决策、本地计算频率和传输功率等多个相互耦合的决策变量，传统凸优化方法难以直接求解。为此，本节采用基于 WoLF-PHC（Win or Learn Fast-Policy Hill-Climbing）的多智能体强化学习算法，将每个工业设备建模为独立的智能体，实现多维决策变量的分布式协同优化，兼顾收敛速度与计算效率。

1.4.3.1 强化学习模型构建

将每个工业设备视为一个独立的智能体，与 MEC 系统环境进行交互。在每个时隙，智能体根据当前策略选择动作，环境根据动作更新状态并返回奖励。智能体的目标是最大化长期累积奖励。

（1）状态空间：每个智能体（设备）的状态需包含系统联合状态与自身局部状态，确保决策的全面性与针对性，定义为：

$$\boldsymbol{s_i}(t) = [Q_i(t), Q_i^X(t), Q_i^Y(t), Q_i^Z(t), \gamma_i(t), \delta_i(t), \mathrm{Pr}_i]. \quad 46.$$

其中，$Q_i(t), Q_i^X(t), Q_i^Y(t), Q_i^Z(t)$为实际队列与虚拟队列长度，反映系统负载状态；$\gamma_i(t)$为带宽分配比例，$\delta_i(t)$为接收信噪比，反映信道与资源分配状态；$\mathrm{Pr}_i$

为任务优先级，反映 QoS 需求特性。系统全局状态为所有智能体状态的集合：$\boldsymbol{s}(t) = [s_1(t), s_2(t), \dots, s_N(t)]$。

（2）动作空间：为避免动作空间维度爆炸，本节结合变量耦合特性进行结构化设计，动作空间为：

$$\mathbf{a_i}(t) = [\theta_i(t), a_i^c(t)], \tag{47.}$$

其中，$\theta_i(t) \in \{0,1\}$为卸载决策。$a_i^c(t)$为连续决策变量，根据$\theta_i(t)$自适应调整，为简化求解，对连续决策变量进行离散化处理。：

- 若$\theta_i(t) = 0$（本地执行），$a_i^c(t) = f_i^l(t)$（本地 CPU 频率），离散化为 4 个等级$f_i^l(t) \in \{0.2F_{i,\max}^l, 0.4F_{i,\max}^l, 0.6F_{i,\max}^l, 0.8F_{i,\max}^l\}$；
- 若$\theta_i(t) = 1$（MEC 卸载），$a_i^c(t) = p_{tx,i}(t)$（发射功率），离散化为 4 个等级：$p_{tx,i}(t) \in \{0.2p_{tx,i}^{\max}, 0.4p_{tx,i}^{\max}, 0.6p_{tx,i}^{\max}, 0.8p_{tx,i}^{\max}\}$。

最终每个智能体的动作空间大小为 $2 \times 4 = 8$，即 $\mathbf{a_i}(t) = \{a_{i,0}(t), a_{i,1}(t), \dots, a_{i,7}(t)\}$，兼顾决策灵活性与计算复杂度。

（3）奖励函数：奖励函数需与($\boldsymbol{P}$-1)中优化目标Θ直接关联，引导智能体最小化Θ，定义为：

$$\boldsymbol{r_i}(t) = \frac{D - \Theta_i(t)}{D}, \tag{48.}$$

其中，D为固定常数（用于归一化奖励值，确保$r_i(t) \in [0,1]$），$\Theta_i(t)$为设备i对应的Θ，$\Theta_i(t)$越小，奖励$r_i(t)$越大，引导智能体做出最优决策。

（4）策略：$\pi(t) = \{\pi_1(t), \dots, \pi_N(t)\}$为系统策略集合，智能体在状态$s_i(t)$下选择动作$a_i(t)$的策略选择概率分布为$\pi_i(t) = \Pr(a_i(t) = a | s_i(t) = s)$。

1.4.3.2　WoLF-PHC 算法核心机制

WoLF-PHC 算法[错误!未找到引用源。错误!未找到引用源。]具有计算复杂度低、收敛速度快、支持分布式计算等优点，能够有效应对高维状态-动作空间带来的挑战。其核心优势在于通过区分智能体的“获胜”与“失败”状态，采用差异化学习速率更新策略，若当前策略表现优于历史平均策略时，采用较小的学习速率以保持策略稳定；若当前策略表现较差时，采用较大的学习速率以加速策略改进。

$Q_i(s_i, a_i)$表示智能体在状态s下选择动作a的长期累积奖励，更新规则为：

$$Q_i^{t+1}(s_i, a_i) = (1 - \alpha_t) Q_i^t(s_i, a_i) + \alpha_t \left(r_i(t) + \gamma \max_{a_i'} Q_i(s_i', a_i') \right), \tag{49.}$$

其中，$\alpha_t \in (0,1)$为学习速率，随迭代次数递减，平衡探索与利用；$\gamma \in (0,1)$为折扣因子，衡量未来奖励的权重；$s_i{}'$为下一状态，$\max_{a_i{}'} Q_i(s_i{}', a_i{}')$为下一状态的最优 Q 值。

为提升联合优化效率，本节引入状态访问计数$C(s)$记录 Q 值更新过程中各状态的出现次数。估计策略π'的更新规则为：

$$\pi_i'(s, a_i) = \pi_i'(s, a_i) + \frac{1}{C(s)}[\pi_i(s, a_i) - \pi_i'(s, a_i)]. \tag{50}$$

通过比较当前策略与估计策略的期望 Q 值，判断策略“获胜”或“失败”：若$\sum_{a_i \in \mathbf{a_i}} \pi_i(s, a_i) Q_i(s, a_i) > \sum_{a_i \in \mathbf{a_i}} \pi_i'(s, a_i) Q_i(s, a_i)$，则判定当前策略为“获胜”状态，否则，判定当前策略为“失败”状态。根据判定结果选择不同的学习速率：获胜时采用较小的学习速率δ_h，失败时采用较大的学习速率δ_l（$\delta_l > \delta_h$）。这种自适应学习速率机制使算法在保持收敛性的同时加快学习进程。

基于 WoLF-PHC 的多智能体联合优化算法如算法 3-2 所示，实现分布式资源调度决策的在线学习与优化。

算法3-2：基于WoLF-PHC的多智能体联合优化算法

输入： 智能体数量N、获胜速率δ_h、失败速率δ_l、学习速率α_t、折扣因子γ、迭代回合数U、初始状态$s(0)$；

输出： 最优策略π_i^*；

1. 对每个智能体i，初始化$Q_i(s_i, a_i) = 0$；初始化策略$\pi_i(s, a_i) = 1/|\mathcal{A}_i|$，$\pi_i{}'(s, a_i) = 1/|\mathcal{A}_i|$；状态访问次数$C(s) = 0$；
2. for episode $= [1, 2, \dots, U]$ do：
3. 　重置状态$s = s(0)$；
4. 　for $t = \{1, 2, \dots, T\}$ do：
5. 　　每个智能体i根据当前策略$\pi_i(s_i(t))$选择动作$a_i(t)$；
6. 　　执行动作$a_i(t)$，依据式 48 获取即时奖励$r_i(t)$，观测下一状态$s'(t)$；
7. 　　每个智能体i按式 49 更新 Q 值$Q_i(s_i, a_i)$；
8. 　　更新状态访问次数$C(s_i(t)) = C(s_i(t)) + 1$；
9. 　　按式 50 更新估计策略$\pi_i{}'(s_i(t), a)$；
10. 　　计算当前策略与估计策略的价值函数，判断“获胜”或“失败”，确定学习速率σ（δ_h或δ_l）；
11. 　　按策略更新规则调整π_i；
12. 　　更新状态$s(t+1) = s'(t)$；

1.5　仿真实验与结果分析

1.5.1　仿真参数设置

为验证本章提出的基于 Lyapunov 优化与 WoLF-PHC 强化学习的低时延高能效资源调度机制的有效性与优越性，本节从 POT 模型适用性、算法收敛性能、系统能耗优化效果及任务时延性能四个核心维度展开仿真实验与分析。

仿真实验考虑单基站场景，基站总无线带宽固定为$BW = 10\text{MHz}$，并在基站侧部署一台 MEC 服务器。系统时隙长度$\tau = 10\,\text{ms}$，仿真总时隙数$T = 10^4$。工业设备随机分布在基站覆盖范围内，所有设备均接入同一 MEC 服务器获取计算卸载服务，MEC 服务器提供集中式计算资源，设备根据自身任务特性与系统状态，在本地执行与 MEC 卸载两种模式间动态选择。为模拟工业业务的异构性，将设备任务划分为三个优先级等级，不同优先级对应差异化的时延阈值与可靠性需求，贴合工业场景中关键实时任务（如故障预警）与非关键实时任务（如环境监测）的 QoS 差异。三种优先级值Pr_i，分为最高、中等、最低三个等级，分别对应不同的时延阈值$t^{max} = \{5,25,500\}$（ms）和可靠性指标$\lambda = \{0.001,0.01,0.1\}$，优先级越高，时延要求越严苛、可靠性需求越高。任务数据量$b_i(t)$服从区间$(300,500)$ kbits的均匀分布，任务计算复杂度$d_i(t)$服从区间$(70,800)$ cycle/bit的均匀分布，模拟工业任务的随机性。除特别说明外，MEC 服务器的最大计算能力设置为$F_{ser} = 10\text{GHz/s}$，任务处理密度L_i设置为 737.5 cycles/bit。各设备的本地 CPU 频率和发射功率均离散化为 4 个等级：$f_i^l = \{20,40,60,80\}$（MHz/s），$p_{tx,i} = \{0.5,1,5,10\}$（W）。如前文所述，GPD 分布可由两个参数完全刻画，其初始值分别设置为$\sigma_i = 6 \times 10^7$ bits 和$\xi_i = -1.75$。权衡参数V设置为4×10^{27}，用于平衡队列稳定性与能耗优化精度，经文献调研和仿真验证[错误!未找到引用源。错误!未找到引用源。错误!未找到引用源。]，该值可实现二者的最优协同。其他详细仿真参数如表 3-1 所示。

表 3-1 仿真参数设置

参数名称	参数取值	参数名称	参数取值
时隙长度 τ	10 ms	信道衰落系数 g_0	-40 dB
设备能耗系数 κ_{mob}	10^{-27}	噪声功率谱密度 N_0	-174 dBm/Hz
服务器能耗系数 κ_{ser}	10^{-27}	队列阈值 q_0	3.96×10^6 kbits
获胜学习速率 δ_h	0.004	处理密度 L_i	737.5 cycles/bit
失败学习速率 δ_l	0.002	任务数据量 $b_i(t)$	(300, 500) kbits
学习速率 α^t	0.100	计算密度 $d_i(t)$	(70, 800) cycles/bit

为全面验证所提 EVT learning 算法的性能优势，选取两种典型的工业任务调度策略作为对比，三种算法的核心逻辑如下，确保对比的公平性与针对性：

（1）Non-MEC 算法：所有工业设备均采用本地计算模式，不进行任务卸载，设备根据自身时延约束调整本地 CPU 频率，仅优化本地计算能耗，未考虑 MEC 资源的利用，代表传统工业设备的独立计算模式；

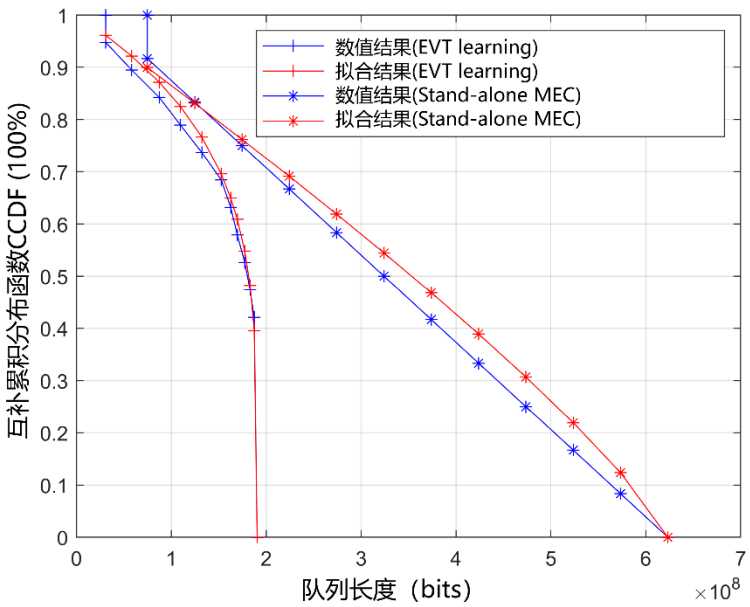
（2）Stand-alone MEC 算法：所有设备均将任务卸载至 MEC 服务器执行，基站无线带宽BW与 MEC 计算资源F_{ser}采用平均分配策略，代表传统 MEC 卸载的均分资源调度模式；

（3）EVT learning 算法（所提方案）：基于 Lyapunov 优化实现约束转化，采用 MLE-SVRGD 估计 GPD 参数，通过 WoLF-PHC 多智能体强化学习实现卸载决策、本地频率与发射功率的联合优化。

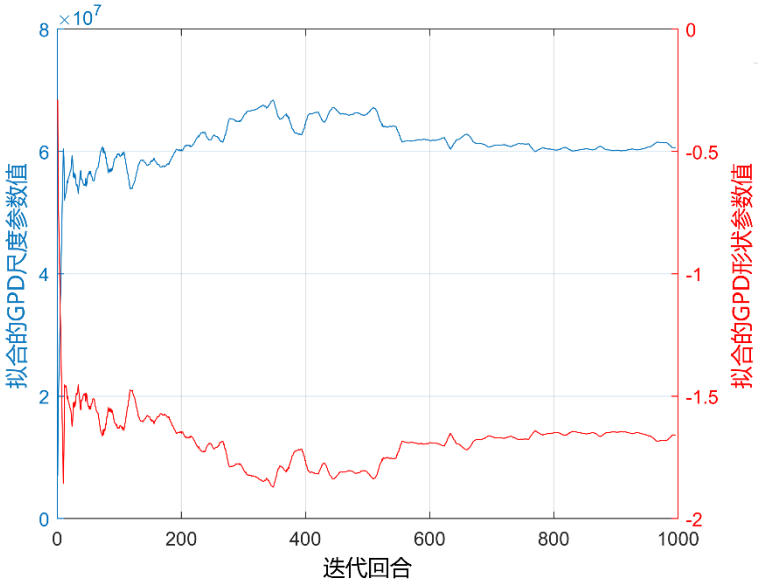
1.5.2 仿真结果分析

1.5.2.1 POT 模型有效性验证

POT 模型作为队列尾部特性管控的核心基础，其有效性直接决定队列溢出概率与长尾时延的管控精度。本实验通过对比 EVT learning 算法与 Stand-alone MEC 算法的超阈值队列长度的互补累积分布函数（Complementary Cumulative Distribution Function, CCDF）及 GPD 理论分布，验证 POT 模型在描述队列长度超阈值特性方面的有效性。



(a) 互补累积分布函数对比



(b) GPD 参数收敛曲线

图 3-3 POT 模型有效性验证

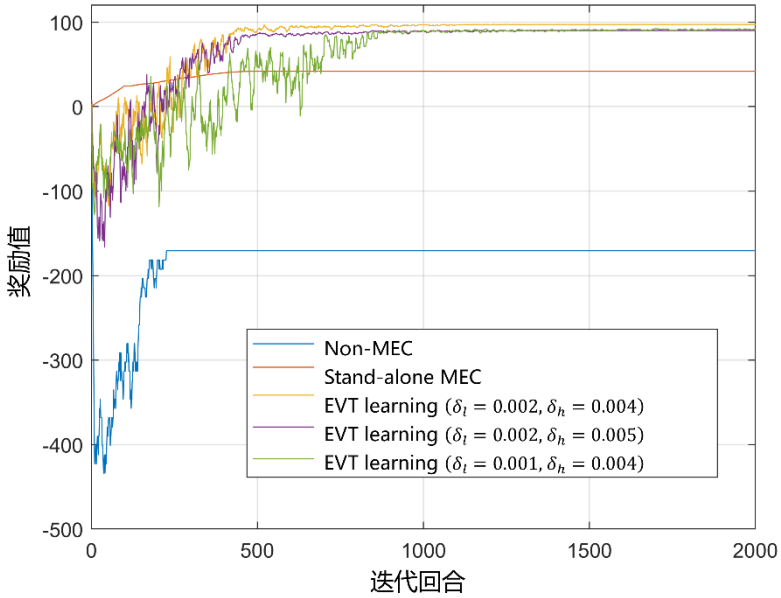
图 3-3(a)给出了两种算法超阈值队列长度的 CCDF 曲线，CCDF 曲线反映了队列长度超过某一阈值的概率，曲线越靠下，说明队列长度超阈值的概率越低，队列稳定性越好。从图中可以看出，所提 EVT learning 算法的 CCDF 曲线整体位于 Stand-alone MEC 算法下方，且超阈值队列长度样本主要集中在队列阈值q_0附近，表明该算法能够有效管控超阈值队列长度的分布，对队列长度的极端值具有良好的刻画与约束能力。而 Stand-alone MEC 算法的超阈值队列长度样本分布分散，无明显集中趋势，无法有效管控队列尾部的极端波动，这是因为该算法未考虑队列尾部特性约束，仅采用平均资源分配策略，易导致 MEC 任务队列积压，进而引发超阈值队列长度的随机波动。从队列长度管控的角度来看，EVT learning 算法通过 POT 模型与 GPD 参数估计，实现了队列优化的确定性，更适用于对时延和可靠性要求严苛的工业关键任务场景。

图 3-3(b)展示了 GPD 模型中尺度参数σ_i与形状参数ξ_i的收敛过程，从图中可以看出，随着仿真时隙的增加，两个参数均逐渐收敛至稳定值，其中尺度参数σ_i收敛至约5.8×10^7 bits，形状参数ξ_i收敛至约-1.72，与初始设置值接近，验证了 1.4.2 节 MLE-SVRGD 参数估计算法的有效性。收敛后的 GPD 参数能够为超阈值队列长度事件的预测提供精准的参考依据，可提前预判队列溢出风险，为工业关键任务的时延管控提供支撑，这对于保障工业业务的连续性与可靠性具有重要意义。

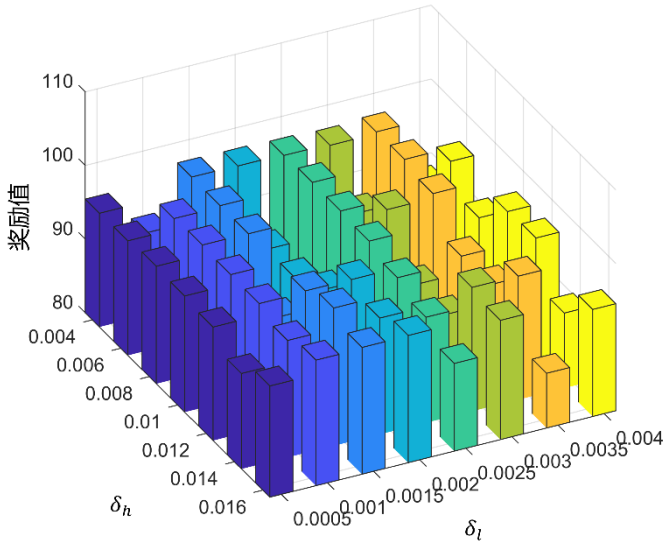
1.5.2.2　WoLF-PHC 算法收敛性能与超参数影响分析

WoLF-PHC 算法的收敛性能直接决定系统调度决策的稳定性与优化精度，而算法超参数（获胜速率δ_h、失败速率δ_l）的设置会影响收敛速度与最终奖励值。

本实验通过分析不同算法的奖励收敛特性及超参数对奖励值的影响，验证 WoLF-PHC 算法的有效性。



(a) 不同策略的奖励收敛曲线



(b) 不同 WoLF-PHC 超参数下的奖励值

图 3-4 算法收敛性能评估

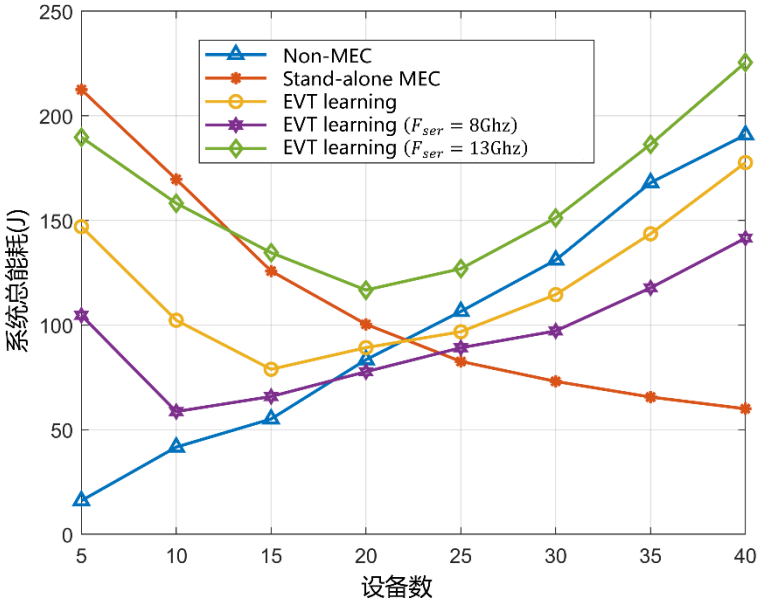
图 3-4(a)对比了三种算法的奖励值收敛过程，值得注意的是，奖励值与时延要求和队列长度密切相关。奖励值越大，表明算法的优化效果越好（即 Θ 越小，能耗与队列约束违反量越低）。从图中可以看出，三种算法的奖励值均能最终收敛，但收敛后的奖励值存在显著差异：EVT learning 算法的最终奖励值最高，约为 0.85；Stand-alone MEC 算法次之，约为 0.68；Non-MEC 算法最低，约为 0.52。造成这种差异的核心原因的是：Non-MEC 算法仅依赖本地计算，部分高优先级

任务因本地 CPU 资源不足无法满足时延约束，导致奖励值降低；Stand-alone MEC 算法采用平均资源分配，易导致 MEC 队列溢出，违反队列约束，进而影响奖励值；而 EVT learning 算法通过联合优化卸载决策与资源分配，兼顾时延约束、队列稳定性与能耗优化，能够有效避免上述问题，因此获得更高的奖励值。

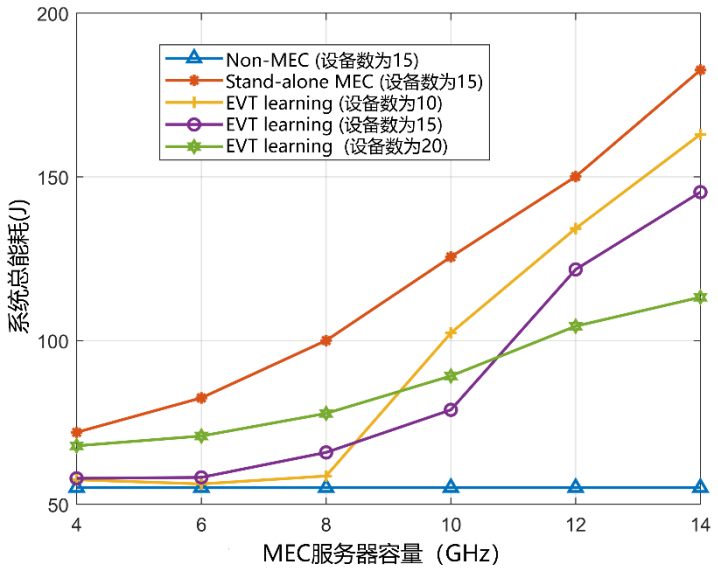
图 3-4(b)探究了 WoLF-PHC 算法中获胜速率δ_h、失败速率δ_l对奖励值的影响，实验设置不同的δ_h、δ_l组合，观察最终奖励值的变化。从图中可以看出，不同超参数组合下的奖励值整体差异不大，但存在明显的优化区间：当失败速率δ_l在 0.001~0.002 范围内时，奖励值更高；而获胜速率δ_h对奖励值的影响相对较小。在相同δ_l下，δ_h越大，奖励值的方差越小，算法的稳定性越好。这是因为 WoLF-PHC 算法的收敛速度主要由失败速率决定，当算法处于“失败”状态时，较快的学习速率能够加速策略调整，快速收敛至更优策略；而获胜速率主要用于稳定“获胜”状态的策略，减少策略波动。结合实验结果，后续所有仿真实验中，WoLF-PHC 算法的超参数设置为$\delta_h = 0.004$、$\delta_l = 0.002$，以实现奖励值与算法稳定性的最优平衡。

1.5.2.3　系统能耗性能对比分析

系统长期时间平均能耗是本章的核心优化目标，本实验从设备数量、MEC 服务器容量和时延阈值三个关键影响因素出发，对比三种算法的能耗性能。



(a) 设备数量对总能耗的影响



(b) MEC 服务器容量对总能耗的影响

图 3-5 系统总能耗与设备数量及 MEC 服务器容量的关系

图 3-5(a)展示了设备数量对系统总能耗的影响，其中 MEC 服务器容量F_{ser}可动态调整，曲线最低点对应该设备数量下的最优 MEC 容量。从图中可以看出，EVT learning 算法的能耗曲线始终低于另外两种对比算法，展现出最优的节能性能：当 UE 数量为 10、15、20 时，EVT learning 算法的能耗最低点分别对应$F_{\mathrm{ser}} = 8\ \mathrm{GHz/s}$、$10\ \mathrm{GHz/s}$、$13\ \mathrm{GHz/s}$，表明算法能够根据设备数量动态匹配最优 MEC 资源，实现能耗与资源利用率的协同优化。当 UE 数量小于曲线最低点对应的设备数时，随着 UE 数量增加，设备更倾向于将任务卸载至 MEC 服务器以满足时延约束；由于 MEC 服务器计算能耗与计算频率的平方成正比，计算能耗的降低量大于传输能耗的增加量，因此系统总能耗呈下降趋势；当 UE 数量超过最低点对应的设备数时，若所有任务均卸载至 MEC 服务器，会导致 MEC 队列积压，此时算法会自适应调整部分设备采用本地执行模式，以提升奖励值，而本地执行的能耗相对较高，且 MEC 服务器分配给单个设备的计算资源增加，进而导致系统总能耗呈上升趋势。

图 3-5(b)展示了 MEC 服务器容量F_{ser}对系统总能耗的影响，实验固定 UE 数量，观察能耗随 MEC 容量的变化规律。从图中可以看出，三种算法的能耗曲线均随 MEC 服务器容量的增加呈先下降后上升的趋势，存在一个最优 MEC 容量，此时系统能耗最低。当 MEC 容量超过最优值后，部分任务会开始采用本地执行模式以最小化系统总能耗，由于不同设备的本地执行比例存在差异，部分设备会被分配更多的本地计算资源，导致能耗增加，这一现象与图 3-5(a)的能耗变化规律一致。值得注意的是，无论 MEC 容量如何变化，EVT learning 算法的能耗始

终低于对比算法，尤其是当 MEC 容量处于最优区间时，能耗优势更为明显，验证了该算法在资源动态匹配与能耗优化方面的优越性。

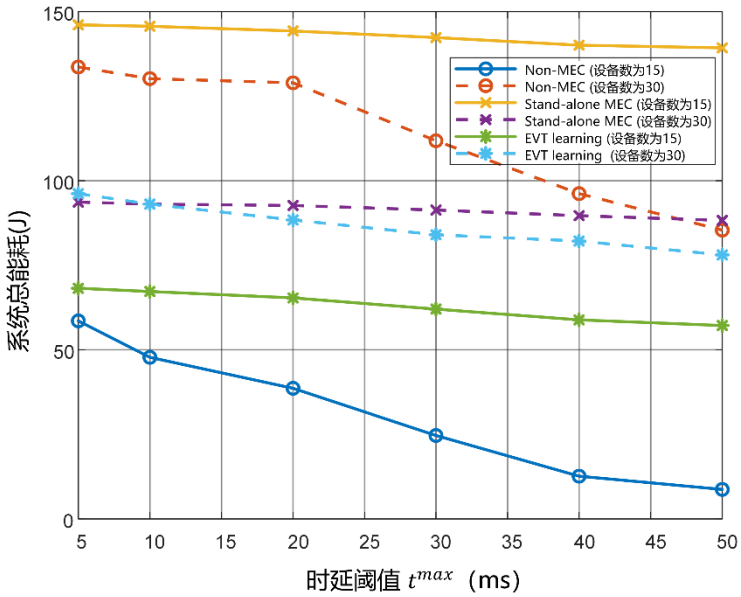


图 3-6 系统能耗与时延阈值的关系

图 3-6 展示了时延阈值对系统总能耗的影响，重点探究高优先级任务（时延阈值$t_{\max} = 5\,\mathrm{ms}$）的时延阈值变化对能耗的影响，由于不同优先级任务的时延阈值变化对系统能耗的影响规律相似，因此本实验仅以高优先级任务为例展开分析。从图中可以看出，随着时延阈值的增加，三种算法的系统总能耗均呈下降趋势，但下降幅度存在显著差异：Non-MEC 算法的能耗下降最为明显，EVT learning 算法次之，Stand-alone MEC 算法的下降幅度最小。这是因为当时延阈值较低时，高优先级任务因本地 CPU 资源限制，无法通过本地执行满足时延约束，Non-MEC 算法只能通过提高本地 CPU 频率来缩短时延，导致能耗激增，因此当时延阈值增加时，能耗下降更为显著；Stand-alone MEC 算法采用平均资源分配，任务执行能耗几乎不受时延阈值影响，时延阈值的增加仅能使设备以更低的发射功率传输任务，因此能耗下降幅度较小；EVT learning 算法能够根据时延阈值动态调整卸载决策与资源分配，在满足时延约束的前提下最小化能耗，因此能耗下降幅度介于两者之间。同时，从图中还可以看出，所提算法能够始终满足任务的可靠性要求，且当设备数量较多时，能耗优势更为突出，进一步验证了算法的实用性。

1.5.2.4　任务时延性能分析

工业场景中，任务优先级是影响调度决策与系统性能的关键因素，本实验探究不同任务优先级下的平均时延性能，验证所提算法的 QoS 保障能力。

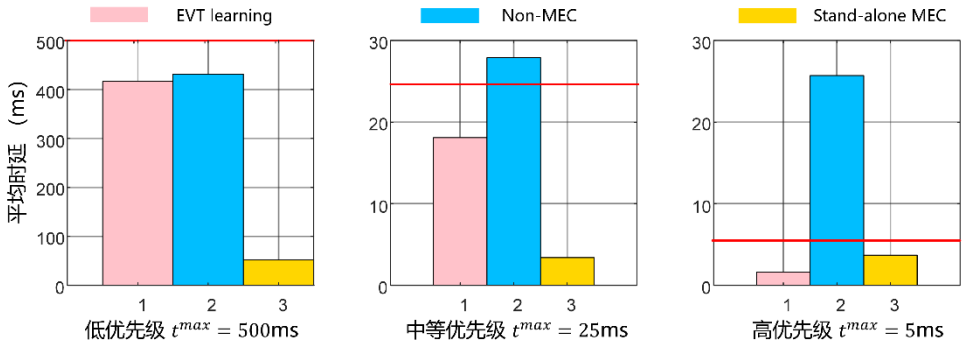


图 3-7 不同服务优先级下的平均时延性能

图 3-7 展示了不同任务优先级下的平均时延性能，从图中可以看出，所提 EVT learning 算法能够满足不同优先级任务的差异化时延需求，体现出良好的 QoS 保障能力：高优先级任务（$t_{\max} = 5$ ms）只能通过卸载至 MEC 服务器执行才能满足时延约束，其平均时延稳定在 3~4 ms，未超过时延阈值；中优先级任务（$t_{\max} = 25$ ms）在时延阈值较小时需通过卸载执行，当时延阈值增加时，算法会根据系统能耗自适应选择本地执行或卸载执行，平均时延稳定在 15~20 ms，满足时延要求；低优先级任务（$t_{\max} = 500$ ms）对时延敏感度较低，算法在确保队列无过度积压的前提下，更倾向于选择本地执行模式，以最小化系统总能耗，其平均时延稳定在 300~400 ms，远低于时延阈值。上述结果表明，EVT learning 算法能够根据任务优先级与时延约束，动态调整调度策略，有效保障工业关键任务的确定性 QoS 需求，同时实现系统能耗优化，适用于差异化动态到达的工业业务场景。

1.6 本章小结

本章围绕工业无线网络中 MEC 边缘计算场景下的确定性服务质量保障问题，针对上行任务随机到达及异构 QoS 需求带来的时延不确定性与能耗优化难题，提出了一种基于极值理论辅助强化学习的低时延高能效资源调度机制。通过引入尾部统计建模与在线学习相结合的优化框架，实现了对任务队列长尾时延的统计约束控制，并在保证服务可靠性的前提下有效降低系统能耗。主要工作与结论总结如下：

（1）构建了基于极值理论的队列尾部风险刻画模型。针对异构工业任务的差异化 QoS 需求，设计了综合考虑时延阈值与可靠性要求的优先级量化方法。引入极值理论中的峰值超阈值（POT）模型，将超过预设阈值的队列长度样本拟合为广义帕累托分布（GPD），通过约束其一阶矩和二阶矩，将不可见的尾部风险转化为可量化、可约束的统计指标，为极端时延事件的概率可控提供了理论依据。

（2）提出了基于 Lyapunov 优化的统计约束转化方法。将 GPD 分布约束与队列溢出概率约束转化为虚拟队列稳定性约束，构建包含实际队列与多个虚拟队列的扩展系统状态。通过定义联合考虑队列积压最小化与能耗惩罚的 Lyapunov 漂移-惩罚函数，推导出其上界表达式，将长期随机优化问题分解为逐时隙可求解的确定性子问题，在保证系统稳定性的前提下显著降低了求解复杂度。

（3）设计了 EVT 嵌入的 WoLF-PHC 多域资源联合调度算法。采用最大似然估计结合随机方差缩减梯度下降技术对 GPD 参数进行在线估计，确保尾部统计约束的动态适应性。针对卸载决策、本地 CPU 频率、传输功率等多维耦合变量，设计基于 WoLF-PHC 的多智能体强化学习算法，通过“获胜”与“失败”状态下的差异化学习速率机制，在保障收敛性能的同时提升学习效率，实现多用户场景下的实时自适应调度。

（4）仿真验证了所提机制的有效性与优越性。结果表明，所提 POT 模型能够准确刻画队列长度的尾部分布特性，为确定性控制提供可靠的统计基础。在保障关键任务时延违约概率受控的前提下，所提算法相较于本地计算策略和传统 MEC 平均分配策略，分别降低系统总能耗约 35%和 22%，并显著抑制了队列长尾时延风险，体现了更优的稳定性与鲁棒性。

本章研究通过将极值理论引入任务队列分析，实现了从“平均性能优化”到“统计确定性保障”的提升，为工业无线网络在随机突发到达场景下的低时延高能效调度提供了可量化、可验证的理论基础。然而，统计约束仅能保证时延在概率意义上满足要求，无法提供严格意义上的时延上下界确定性保障。此外，本章聚焦于任务卸载场景下的计算与通信资源协同优化，尚未考虑当上行带宽资源受限时，单纯依赖传统比特传输模式可能出现的传输瓶颈问题。当工业设备密集接入且业务流量持续增长时，有限带宽资源将成为制约系统性能的关键因素，亟需探索超越传统比特传输的新型通信范式。针对上述挑战，下一章将研究面向上行带宽受限场景的基于语义压缩的低时延低功耗资源优化机制，通过引入语义通信技术突破传统比特级传输的性能瓶颈，在保证控制性能的同时进一步提升系统能效。

第二章